

3、大模型预训练与微调 完整流程

LLM Training · PyTorch · Transformer · Pretraining · SFT · LoRA · QLoRA · DPO · Deployment

最基础的概念开始，一直讲到如何真正训练、微调、评估和部署一个大语言模型（LLM）。

学习目标： 学完之后，你应该能理解大模型训练的完整技术链路，知道每个环节的作用、关键参数如何设置，以及如何使用 PyTorch、Transformers、PEFT 和 TRL 实现训练。



一、先理解：大模型究竟是怎么训练出来的？

大模型并不是一开始就能聊天、写代码和回答问题的。

一个典型的语言模型，需要经历几个不同阶段。

大模型从数据到上线的完整生命周期

</> HTML 块 [该内容不支持导出查看]

这里有一个重要区别：

- **预训练**：让模型通过海量数据学习语言、知识与模式。
- **持续预训练**：在已有基础模型上进一步学习特定领域的数据分布。如code、网络安全等
- **监督微调（SFT）**：教模型根据指令生成符合要求的答案。
- **偏好对齐**：让模型更倾向于生成被认为更好的回答。
- **部署**：将训练好的模型变成可以提供服务的应用。

不是每个项目都必须完成所有阶段。尤其是企业应用，通常可以直接从已有开源模型的微调开始。

二、预训练和微调究竟有什么区别？

对比维度	预训练 Pretraining	微调 Fine-tuning
初始模型	通常随机初始化	已经训练过的模型
数据	大规模文本和代码等	特定任务、领域或指令数据
目标	学习通用语言分布与能力	优化具体能力或行为
参数更新	通常全部参数	全参数或部分参数
计算资源	通常非常高	相对较低
产物	基础模型 Base Model	任务模型或指令模型
常见应用	训练自己的通用 LLM	客服、代码助手、领域问答

举个例子：

预训练数据：

代码块

```
机器学习是人工智能的一个分支。
深度学习使用多层神经网络进行表示学习。
Transformer 是一种基于注意力机制的神经网络架构。
```

模型的任务不是直接回答问题，而是根据前面的文本预测后面的 Token。

微调数据：

代码块

```
{
  "instruction": "什么是机器学习？",
  "output": "机器学习是让计算机通过数据学习规律，并利用这些规律完成预测或决策的方法。"
}
```

模型开始学习：

用户问什么 → 应该怎么回答。

预训练并非只负责知识，微调也并非只负责格式；两者都可能改变模型的知识和能力，但训练目标与数据分布不同。

三、正式训练前：先确定训练目标和技术路线

这是实际项目中非常重要、却经常被忽略的一步。

假设现在有四个需求：

需求	更合适的方案	原因
从零开发一个通用大语言模型	Pretraining	需要从随机权重开始学习
让现有模型更熟悉医疗、金融、法律等领域	CPT，必要时加 SFT	需要适应特定领域的数据分布
让模型按照指定格式回答客服问题	SFT / LoRA	核心是改变模型的响应行为
让模型查询经常更新的企业知识库	RAG，必要时加 SFT	知识更新不必每次重新训练模型

特别注意：不是所有问题都应该通过微调解决。

如果希望模型记住一份每天更新的产品价格表，通常更适合使用 RAG（检索增强生成）而不是不断微调。

如果希望模型稳定输出固定 JSON 格式、掌握特定任务步骤或遵循行业术语，则可以考虑微调。

接下来按照真正从零训练大模型的路线展开。

四、第一阶段：训练数据准备

模型训练最先接触的不是神经网络，而是数据。

在实际的大规模预训练项目中，数据工程可能占据大量研发时间。

4.1 收集原始数据

常见数据来源：

数据类型	示例	主要作用
网页文本	公开文章、百科内容	通用知识、语言表达
书籍	合法授权的书籍	长文本理解、叙事和知识
学术资料	论文、技术文档	专业知识
代码	许可允许使用的代码仓库	编程能力
数学文本	解题过程、公式推导	数学与符号处理
领域文档	产品手册、行业规范	特定领域能力
对话数据	人工编写的多轮问答	主要用于指令微调

必须考虑数据使用许可、隐私、安全、保密要求，不应把没有授权的内部数据或含敏感信息的原始记录直接送入训练流程。

4.2 数据清洗

假设拿到了 1000 万篇网页文档。

原始文档可能包含：

```
代码块
1  首页 > 新闻 > 技术
2  广告：点击领取优惠！
3  用户登录 | 注册
4
5  Transformer 是一种神经网络架构，
6  它通过注意力机制处理序列信息。
7
8  相关阅读 | 网站声明 | 联系我们
```

真正希望模型学习的主要内容是：

```
代码块
1  Transformer 是一种神经网络架构，
2  它通过注意力机制处理序列信息。
```

因此数据清洗通常包含：

1. 内容抽取：从 HTML、PDF 等格式提取文本，尽量保留标题、段落、代码等结构。
2. 质量过滤：去除空白、乱码、广告模板、机器重复内容。
3. 语言识别：识别中文、英文等，控制各类语料比例。
4. 精确去重：对文档计算哈希，清除完全重复文本。
5. 近似去重：通过 MinHash、LSH 等方法识别高度相似文档。
6. 隐私与合规处理：识别并处理个人信息、秘密、无许可数据等。
7. 数据污染检测：避免训练数据包含本应独立评估的测试答案。
8. 质量抽检：人工查看样本，确认过滤规则没有误删有价值的内容。

4.3 数据集划分

一般需要三部分：

示例：按文档分组划分数据集

Train	Validation	Test
90%	5%	5%

这个比例仅为示例，不是固定标准。

- Train：用于梯度更新。
- Validation：用于超参数选择、训练监控与早停决策。
- Test：用于最终独立评估，不应反复用它调参。

关键原则： 先按文档、来源或其他合理分组划分，再切成 Token 片段，避免同一篇文章的前半部分进入训练集、后半部分进入测试集，导致评估结果虚高。

五、第二阶段：Tokenizer 训练

计算机无法直接把一篇中文文章送入 Transformer。

需要先将文本转换为 Token ID。

5.1 什么是 Token?

例如：

原始文本：

```
代码块
1   人工智能正在改变世界
```

Tokenizer 可能将它拆分成：

```
代码块
1   ["人工", "智能", "正在", "改变", "世界"]
```

再映射为数字：

```
代码块
1   [1532, 2876, 592, 1042, 3180]
```

以上切分和编号只是示意，真实结果取决于 Tokenizer。

模型接收的是数字序列，而不是直接接收文字。

5.2 为什么不直接按字切分?

因为不同切分策略影响计算效率。

例如:

```
代码块
1   "machine learning"
```

不同 Tokenizer 可能切分为:

```
代码块
1   ["machine", "learning"]
2
3   ["mach", "ine", "learn", "ing"]
4
5   ["m", "a", "c", "h", ...]
```

如果切得太细,相同文本需要更多 Token,训练和推理通常就需要更多计算。

如果词表过大,则词嵌入层和输出层的存储、计算成本可能增加。

常见 Tokenizer 算法包括:

- BPE(Byte Pair Encoding)
- WordPiece
- Unigram

训练自己的基础模型时,可以训练一个新 Tokenizer;但微调现有模型时,通常应继续使用原模型的 Tokenizer,避免 Token ID 与已训练词嵌入不匹配。

来源:Hugging Face

5.3 Tokenizer 示例

```
代码块
1   from tokenizers import Tokenizer, models, trainers, pre_tokenizers
2
3   # 创建 BPE Tokenizer
4   tokenizer = Tokenizer(models.BPE(unk_token="[UNK]"))
5
6   # 以字节为基础进行预分词
7   tokenizer.pre_tokenizer = pre_tokenizers.ByteLevel()
8
```

```
trainer = trainers.BpeTrainer(
    vocab_size=8000,
    min_frequency=2,
    special_tokens=["[UNK]", "[PAD]", "[BOS]", "[EOS]"]
)

# 从训练文本中学习词表
tokenizer.train(["train.txt"], trainer)

# 保存 Tokenizer
tokenizer.save("tokenizer.json")

# 编码
result = tokenizer.encode("人工智能正在改变世界")
print(result.ids)
```

这里的 train.txt 是你准备好的语料文件，8000 是教学用词表大小。实际通用大模型的词表通常更大。

六、第三阶段：构建 Transformer 模型

现在已经有了 Token ID，下一步是构建神经网络。

本文主要讲现代大语言模型常用的 Decoder-only Transformer（仅解码器架构）。

6.1 Transformer 的核心结构

</> HTML 块 [该内容不支持导出查看]

这是概念简图。不同模型的归一化顺序、注意力结构、MLP 结构和位置编码方式可能不同，例如现代架构常使用 RMSNorm、RoPE、GQA、SwiGLU 等。

6.2 Embedding 层

假设：

- 词表大小：50,000
- 隐藏维度：768

那么词嵌入矩阵形状为：

$$E \in \mathbb{R}^{50000 \times 768}$$

它会把 Token ID 映射成 768 维向量。

例如：

```
代码块
```

```
Token ID: 1532

Embedding:

[0.12, -0.32, 0.58, ..., 0.07]
```

注意，刚开始这些向量没有经过训练，基本不包含有用的语言信息。

6.3 Self-Attention：注意力机制

这是 Transformer 的核心。

假设句子是：

小明拿起水杯，因为他口渴了。

模型处理“他”时，需要从上下文理解它与“小明”的联系。

注意力机制让不同位置的 Token 进行信息交互。

核心公式：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}+M\right)V$$

其中：

- Q：Query，查询向量。
- K：Key，键向量。
- V：Value，值向量。
- d_k：Key 的维度。
- M：注意力掩码。

Decoder-only 模型使用因果掩码（Causal Mask），阻止当前位置看到未来 Token。

举个例子：

因果注意力矩阵：✓ 可关注，— 不可关注

当前位置	我	喜欢	学习	AI
我	✓	—	—	—
喜欢	✓	✓	—	—
学习	✓	✓	✓	—
AI	✓	✓	✓	✓

注意力不是简单地从句子中找某个词，而是在不同表示之间学习信息关系。

6.4 模型参数量是怎么来的?

参数主要来自：

- Token Embedding
- Attention 中的线性变换矩阵
- MLP 中的线性变换矩阵
- 归一化参数
- 输出投影层（有些模型与 Embedding 权重共享）

模型规模受这些超参数影响：

参数	含义
`vocab_size`	词表大小
`hidden_size`	隐藏层维度
`num_hidden_layers`	Transformer 层数
`num_attention_heads`	注意力头数量
`intermediate_size`	MLP 中间维度
`max_position_embeddings`	位置编码相关配置，具体取决于架构

要区分：

7B 参数通常意味着模型约有 70 亿个可学习参数，并不是指使用了 70 亿条训练数据。

七、第四阶段：理解预训练目标和 Loss

预训练时，模型到底在做什么?

对于 GPT 类的因果语言模型，一个最核心的任务就是：

Next Token Prediction：预测下一个 Token。

7.1 训练样本如何构造?

假设原始 Token 序列是：

```
代码块
```

```
[我，喜欢，学习，人工智能]
```

可以理解为：

输入上下文	预测目标
我	喜欢
我 喜欢	学习
我 喜欢 学习	人工智能

实际训练会并行计算序列中多个位置的预测损失，而不是每预测一个 Token 就单独跑一次整个模型。

数学上：

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P_\theta(x_t \mid x_{<t})$$

对应的交叉熵损失（忽略被掩码的位置）：

$$\mathcal{L} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P_\theta(x_{t+1} \mid x_{\leq t})$$

可以理解为：

- 预测正确 Token 的概率越高，Loss 通常越低。
- 预测错误或给正确 Token 的概率过低，Loss 通常越高。

7.2 一次训练更新究竟发生了什么？

1. **取出一个 Batch**
   从训练语料获取一批 Token 序列。
2. **Forward 前向传播**
   Token 经过 Embedding、Transformer 各层，得到预测 Logits。
3. **计算 Loss**
   使用 Cross Entropy 衡量预测 Token 与真实 Token 的差异。
4. **Backward 反向传播**
   通过链式法则计算参数梯度。
5. **Optimizer 更新**
   使用 AdamW 等优化器更新可训练参数。

6. **重复训练**
   经过大量 Step，让模型逐步学习数据中的规律。

最基本的 PyTorch 训练循环如下：

代码块

```
optimizer.zero_grad()

for batch in train_loader:
    input_ids = batch["input_ids"].to(device)
    labels = batch["labels"].to(device)

    # 1. 前向传播
    outputs = model(
        input_ids=input_ids,
        labels=labels
    )

    # 2. 计算损失
    loss = outputs.loss

    # 3. 反向传播
    loss.backward()

    # 4. 梯度裁剪
    torch.nn.utils.clip_grad_norm_(
        model.parameters(), 1.0
    )

    # 5. 更新参数
    optimizer.step()

    # 6. 清空梯度
    optimizer.zero_grad()
```

这是教学版循环：真实项目还要处理学习率调度、梯度累积、混合精度、分布式同步、异常恢复和检查点保存等。

在 Hugging Face 的 Causal LM 实现中，传入 `labels=input_ids` 时，模型内部通常会完成下一个 Token 预测所需的位置偏移，不需要手动将标签再偏移一次。

来源：Hugging Face

八、第五阶段：正式预训练的关键超参数

8.1 Batch Size、Epoch、Step 的区别

这三个概念一定要理解。

Batch Size：一次训练处理多少条样本序列。

Epoch：完整遍历一遍指定训练集。

Optimizer Step：优化器真正更新一次参数。

在大规模预训练中，通常更关心训练的总 Token 数与更新步数，而不只看 Epoch。

有效 Batch Size

多 GPU、梯度累积时：

$$B_{\text{effective}} = B_{\text{micro}} \times G \times A$$

其中：

- B_{micro}：每张 GPU、每次前向的序列数。
- G：参与数据并行的 GPU 数量。
- A：梯度累积次数。

举个例子：

</> HTML 块 [该内容不支持导出查看]

如果每条序列有 2048 个有效 Token，那么一次优化器更新大约处理：

$$128 \times 2048 = 262144 \ \text{Tokens}$$

这也是规划训练成本时非常重要的指标。

8.2 常见训练参数

参数	作用	教学起点
Learning Rate	参数更新幅度	根据规模和训练方式调整
Warmup	初期逐步提高学习率	总步数约 1%–5% 可作为实验起点
Optimizer	参数优化方式	AdamW
Weight Decay	权重衰减	如 0.01–0.1
Gradient Clipping	限制过大的梯度	如 1.0
Scheduler	调整学习率曲线	Cosine 或其他策略
Context Length	单条训练序列最大长度	如 1024、2048、4096

Seed	控制随机性	如 42

这些都是调参起点，不是适用于全部模型的最优值。预训练、全参数微调和 LoRA 的学习率通常会有明显差异。

8.3 梯度累积解决什么问题？

假设 GPU 显存只能容纳 2 条序列，但你想让一次更新相当于训练 16 条序列。

可以连续执行 8 次小 Batch 的前向、反向传播，最后才更新参数。

代码块

```
grad_accum_steps = 8
optimizer.zero_grad()

for step, batch in enumerate(train_loader):
    outputs = model(**batch)
    loss = outputs.loss / grad_accum_steps
    loss.backward()

    if (step + 1) % grad_accum_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

这里是简化示例。真实代码还必须处理最后不足 8 个 Micro Batch 的情况；如果各 Micro Batch 的有效 Token 数差异较大，还应正确处理按 Token 加权的损失归一化。

梯度累积能够降低单次前向传播的显存需求，但不意味着训练总计算量减少。

8.4 如何估算预训练计算量？

对于常见 Dense Decoder-only Transformer，可以用一个粗略公式：

$$\text{Training FLOPs} \approx 6ND$$

其中：

- N：模型参数量。
- D：训练 Token 数量。

假设：

- 模型：10 亿参数（1B）
- 数据：200 亿 Token（20B）

那么：

$$6 \times 10^9 \times 2 \times 10^{10} = 1.2 \times 10^{20} \text{ FLOPs}$$

这只是粗略估计，不等于实际训练时间。真实成本还受到上下文长度、注意力实现、设备利用率、通信效率、数据加载速度等因素影响。

九、第六阶段：显存、GPU 和分布式训练

前面的循环在小模型上容易执行，但真正的大模型预训练会遇到一个核心问题：

模型参数、梯度、优化器状态和中间激活，可能远远超过单张 GPU 的显存。

9.1 为什么一个 7B 模型训练起来需要那么多显存？

假设模型有 70 亿参数。

如果只使用 BF16 保存模型权重：

$$7 \times 10^9 \times 2 \text{ Bytes} \approx 14 \text{ GB}$$

这只是模型权重，并不包含完整训练所需的其他内容。

以一种常见的混合精度 AdamW 全参数训练配置为例，每个参数可能涉及：

内容	近似字节/参数
BF16 参数	2
FP32 主权重副本	4
梯度	2 或 4
Adam 一阶状态	4
Adam 二阶状态	4

仅这些常驻状态就可能达到约 16–18 Bytes/参数，而且还没算激活、临时张量和通信缓存。具体取决于优化器及精度实现。

因此，不能因为 7B 模型 BF16 权重大约 14GB，就认为 16GB 显卡能完成它的全参数训练。

9.2 常见训练方案与资源需求

以下只是规划时的粗略参考，不能视为硬件兼容性保证。

训练任务	大致资源级别
从零训练数百万参数教学模型	CPU 或普通 GPU

0.5B–1B 模型 LoRA	单张中等显存 GPU 通常可以尝试
7B QLoRA，较短上下文	常可在约 16–24GB 或更高显存设备上尝试
7B 全参数微调	通常需要高显存 GPU、多卡或显存卸载等技术
7B 以上模型大规模预训练	通常需要多 GPU 分布式系统
数十至数百 B 模型预训练	大规模集群及成熟的训练基础设施

实际显存用量还由序列长度、Micro Batch、精度、Attention 实现和优化器决定。

9.3 分布式训练的核心技术

技术	核心原理	解决的问题
DDP	多 GPU 各自保存模型副本，进行梯度同步	提高数据并行吞吐量
FSDP	将模型状态分片保存到不同 GPU	减少单卡模型状态显存
ZeRO	分片优化器、梯度和参数状态	优化显存利用
TP	把单个层的张量计算拆到多 GPU	单层无法放入一张 GPU
PP	将不同模型层分布到不同设备	模型深度方向的切分
EP	将 MoE 专家分配到不同设备	支持稀疏专家模型
Activation Checkpointing	反向传播时重算中间激活	用额外计算换显存
FlashAttention	优化注意力计算和访存	降低注意力相关开销

例如在多 GPU 下启动 Trainer 风格的脚本，可以使用：

代码块

```
accelerate config
accelerate launch train.py
```

实际分片策略需要在 Accelerate/FSDP 等配置中指定，不能仅依靠这条命令就自动获得所有分布式能力。

来源：Hugging Face

9.4 正式预训练还需要哪些工程能力？

真实工业训练平台还需要：

- 数据流式加载、数据分片与多进程预取
- 混合精度（BF16 / FP16）
- 分布式检查点保存和恢复
- 梯度范数、Loss、学习率及吞吐量监控
- 数据版本、模型版本和实验配置管理
- NaN/Inf、GPU 故障及节点中断检测
- 数据吞吐量与 GPU 利用率优化
- 周期性模型评估与训练停止策略

例如你训练到了第 30,000 Step，突然机器故障。

这时应该能从最近的 Checkpoint 恢复模型参数、优化器状态、学习率调度器、随机数状态以及数据消费进度，而不是完全重新训练。

十、第七阶段：预训练结果如何判断？

最基础的指标是训练 Loss 和验证 Loss。

示意：训练与验证 Loss

</> HTML 块 [该内容不支持导出查看]

人为构造的曲线，仅用于解释，不是真实训练实验。

这个例子说明：

前半段：

- 训练 Loss 下降。
- 验证 Loss 也下降。
- 模型在学习，并且对未见样本的预测有所改善。

后半段：

- 训练 Loss 继续下降。
- 验证 Loss 反而上升。
- 可能出现过拟合，也应排查训练与验证数据分布是否存在差异。

对于语言模型，还可以计算：

$$PPL = \exp(\mathrm{CrossEntropyLoss})$$

PPL（Perplexity）越低，通常表示在相同评估设置下模型对该数据的预测能力越好。

但 PPL 不能直接等同于真实问答能力，跨 Tokenizer、跨不同评估语料比较时尤其要谨慎。

十一、第八阶段：持续预训练 CPT

假设已经有一个预训练好的通用模型，但你希望它更熟悉金融领域。

这时不一定要从头训练模型。

可以进行：

Continued Pretraining（持续预训练），也叫领域继续预训练或 Domain-Adaptive Pretraining。

11.1 CPT 数据长什么样？

通常还是纯文本：

```
代码块
1  资产负债表是反映企业在特定日期财务状况的报表。
2
3  企业现金流量表分为经营活动、投资活动和筹资活动。
4
5  流动比率是流动资产与流动负债之间的比值。
```

模型继续训练下一个 Token 预测目标。

和从零预训练不同：

```
代码块
1  # 从零预训练：随机初始化权重
2  model = GPT2LMHeadModel(config)
3
4  # 持续预训练：加载已有权重
5  model = AutoModelForCausalLM.from_pretrained(
6      "已有基础模型"
7  )
```

后面的 Forward、Loss、Backward、Optimizer 的基础机制是相似的。

11.2 CPT 的风险

如果仅使用非常狭窄的行业语料长时间训练，模型可能出现：

- 通用能力下降
- 对少量领域内容过度记忆
- 对其他语言或任务的性能退化
- 原有指令遵循能力受损（如果从指令模型继续训练）

所以实际项目往往需要控制领域语料和通用语料的比例，并同时评估领域能力与通用能力。

CPT 完成后，通常仍可继续进行 SFT。

十二、第九阶段：监督微调 SFT（重点）

接下来进入日常企业大模型开发中应用非常广泛的环节。

12.1 SFT 到底在训练什么？

预训练时：

```
代码块
1   输入：Python 是一种
2   目标：编程语言
```

SFT 时：

```
代码块
1   用户：请写一个 Python 冒泡排序。
2
3   助手：
4   def bubble_sort(arr):
5       ...
```

SFT 仍然可以使用 Next Token Prediction 和交叉熵损失。

真正的变化主要在于：

1. 模型从已有权重开始。
2. 数据被组织成特定的任务或对话格式。
3. 训练目标通常只覆盖需要模型生成的部分。
4. 希望模型学会按照任务要求回答，而不只是自由续写。

12.2 推荐的数据结构

实际训练中通常使用 Chat Template，对系统、用户和助手角色进行区分。

例如 JSON 数据：

```
代码块
1   {
2     "messages": [
```

```
      {
        "role": "system",
        "content": "你是一名专业的AI工程师。"
      },
      {
        "role": "user",
        "content": "解释什么是梯度下降。"
      },
      {
        "role": "assistant",
        "content": "梯度下降是一种通过梯度方向迭代优化模型参数的方法。"
      }
    ]
  }
```

也可以使用 prompt + completion 格式：

代码块

```
{
  "prompt": [
    {
      "role": "user",
      "content": "什么是LoRA？"
    }
  ],
  "completion": [
    {
      "role": "assistant",
      "content": "LoRA是一种参数高效微调技术。"
    }
  ]
}
```

当前 TRL 的 SFTTrainer 支持这类数据结构，且对 prompt-completion 格式可以只在 Completion 的 Token 上计算 Loss。

来源：Hugging Face

12.3 为什么需要 Loss Mask?

假设一条训练数据序列是：

代码块

```
[System] 你是一个助手
```

```
[User] 什么是AI?
[Assistant] AI是人工智能。
```

SFT 时我们一般更关心模型生成助手回答的能力。

因此可以让：

代码块

```
System Token     -> 不计入 Loss
User Token       -> 不计入 Loss
Assistant Token  -> 参与 Loss
```

对应的标签类似：

代码块

```
labels = [
    -100, -100, -100, -100,
    312, 987, 1256, 42
]
```

这里 -100 表示在 PyTorch 的交叉熵损失中忽略对应位置。

这能够避免将所有提示文本都当作模型需要主动生成的目标。

不过不同 SFT 训练方案可能使用不同的 Loss Mask 策略，不能认为所有微调都会自动屏蔽非助手 Token。

12.4 SFT 参数参考

下面是 LoRA SFT 时可以尝试的一组初始配置：

参数	示例
Epoch	1–3
Learning Rate	1e-4
Max Length	1024
Micro Batch Size	1–2
Gradient Accumulation	8
LoRA Rank	16

LoRA Alpha	32
LoRA Dropout	0.05
Weight Decay	0.01
Precision	BF16（硬件支持时）

如果是全参数微调，一般会尝试更小的学习率，例如 1e-5 量级，而不是直接照搬 LoRA 的学习率。最终仍应由验证集决定。

十三、第十阶段：全参数微调、LoRA 与 QLoRA

这是实践中非常重要的一章。

同样是微调大模型，可以有不同方式。

13.1 三种方法的区别

方法	更新内容	显存压力	特点
Full Fine-tuning	全部或几乎全部模型参数	高	自由度大，资源需求高
LoRA	训练额外的低秩矩阵	较低	高效、容易保存多个适配器
QLoRA	量化基础权重 + 训练 LoRA	更低	适合显存有限的场景

新手实践建议：先学 LoRA，再学 QLoRA，最后研究全参数微调与大规模预训练。

13.2 LoRA 的数学原理

原本一个线性层：

$$y = Wx$$

全参数微调会更新矩阵 W 。

LoRA 则冻结原有的 W ，增加两个低秩矩阵：

$$W' = W + \frac{\alpha}{r} BA$$

其中：

- W ：原始预训练权重，通常冻结。
- A ：低秩矩阵。

- B：低秩矩阵。
- r：低秩维度（Rank）。
- α：缩放系数。

LoRA 如何改变一个线性层？

组成	状态
原始权重	冻结，不更新
低秩增量 BA	训练新增参数

实际计算：$Wx + (\alpha/r)BAx$

假设原始权重矩阵是：

$W \in \mathbb{R}^{4096\times4096}$

原始矩阵参数数量：

$4096 \times 4096 = 16,777,216$

如果 LoRA Rank = 16：

$A \in \mathbb{R}^{16\times4096}$

$B \in \mathbb{R}^{4096\times16}$

两个矩阵总参数：

$16 \times 4096 + 4096 \times 16 = 131072$

项目	参数量
原始矩阵参数	1677.7 万
LoRA 新增参数	13.1 万

对这个线性层而言，新增的可训练参数只有原始矩阵参数的约 0.78%。

当然，整个模型的 LoRA 参数占比还取决于哪些层被适配。

13.3 LoRA 核心代码

使用 Hugging Face PEFT：

代码块

```
1   from peft import LoraConfig, get_peft_model
```

```
2
3  lora_config = LoraConfig(
4      r=16,
5      lora_alpha=32,
6      lora_dropout=0.05,
7      bias="none",
8      task_type="CAUSAL_LM",
9      target_modules="all-linear"
10 )
11
12 model = get_peft_model(model, lora_config)
13
14 # 查看可训练参数
15 model.print_trainable_parameters()
```

`target_modules="all-linear"` 可以让 PEFT 对支持的线性层应用 LoRA；如果仅想微调注意力相关层，可以针对模型结构指定 `q_proj`、`v_proj` 等模块名称。

这些名称依赖实际模型架构，不能保证不同模型通用。

来源：GitHub

13.4 QLoRA 又是什么？

QLoRA 的关键是：

1. 将预训练基础模型以低位量化形式加载，例如 4-bit。
2. 冻结量化后的基础权重。
3. 添加 LoRA 适配器。
4. 使用梯度更新适配器参数。

常见 4-bit 配置：

代码块

```
1  import torch
2  from transformers import BitsAndBytesConfig
3
4  quant_config = BitsAndBytesConfig(
5      load_in_4bit=True,
6      bnb_4bit_quant_type="nf4",
7      bnb_4bit_compute_dtype=torch.bfloat16,
8      bnb_4bit_use_double_quant=True
9  )
```

再加载模型：

代码块

```
from transformers import AutoModelForCausalLM
from peft import prepare_model_for_kbit_training

model = AutoModelForCausalLM.from_pretrained(
    "你的基础模型",
    quantization_config=quant_config,
    device_map={"": 0}
)
model = prepare_model_for_kbit_training(model)
```

接下来再配置 LoRA 适配器和训练器。

这是单 GPU CUDA 环境的示意，必须安装适用的 bitsandbytes 并确认 GPU 支持相应计算精度。QLoRA 不等于让所有训练计算都变成 4-bit。

来源：Hugging Face

十五、第十一阶段：偏好对齐、DPO 与强化学习

SFT 之后，模型可能已经能够按要求回答问题。

但同一个问题可能有多个答案，质量并不一样。

例如：

用户：
请解释什么是梯度下降。

回答 A：
梯度下降是一种优化算法。
它通过计算损失函数对参数的梯度，
沿着使损失减小的方向迭代更新参数。

回答 B：
梯度下降就是让数字变小。

我们希望模型更倾向于回答 A。

这就是偏好优化可以发挥作用的地方。

15.1 DPO 数据格式

DPO（Direct Preference Optimization）通常使用三部分：

- prompt：用户问题

- chosen：更受偏好的答案
- rejected：较不受偏好的答案

代码块

```
{
  "prompt": "解释什么是梯度下降。",
  "chosen": "梯度下降是一种利用梯度迭代优化参数的方法。",
  "rejected": "梯度下降就是让数字一直变小。"
}
```

与 SFT 只学习目标答案不同，DPO 使用偏好比较信息，优化模型对不同回答的相对倾向，并通过参考模型控制偏离程度。

来源：Hugging Face

15.2 DPO 核心代码

代码块

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    output_dir="./dpo_output",
    learning_rate=5e-6,
    beta=0.1,
    num_train_epochs=1
)

trainer = DPOTrainer(
    model=sft_model,
    args=config,
    train_dataset=preference_dataset
)

trainer.train()
```

这里假定 sft_model 已正确加载，并且 preference_dataset 符合 DPO 数据格式。这是接在 SFT 后面的示意代码，并非可独立运行的完整脚本。学习率与 Beta 需要实验验证。

15.3 RLHF、DPO、GRPO 的区别

方法	主要思想	关键特点

RLHF（经典奖励模型 + PPO 路线）	训练奖励模型，再通过强化学习优化策略	工程链路较复杂
DPO	直接利用偏好数据优化语言模型	不需要单独训练显式奖励模型
GRPO	对一组候选输出使用相对奖励信号进行策略优化	常用于具有可评估奖励的训练场景

并不是 DPO 和 GRPO 越多越好。

比如，如果用户只要求模型按照公司规定返回 JSON，那么高质量 SFT 可能已经足够。

而数学、代码等具有可验证结果的任务，可能适合探索基于可验证奖励的强化学习训练。

十六、第十二阶段：模型评估与验收

真实项目不能只看训练 Loss。

假设你微调一个企业客服模型，微调后的训练 Loss 下降了 30%。

这并不能证明它已经可以上线。

还需要检查模型在未见问题上的实际表现。

16.1 评估指标体系

维度	可以采用的指标
语言建模	Validation Loss、Perplexity
事实问答	Accuracy、Exact Match、F1
指令遵循	指令满足率、格式通过率
结构化输出	JSON Schema 校验通过率
代码生成	单元测试通过率、pass@k
用户偏好	盲测胜率、成对比较
安全性	不当输出率、隐私泄漏测试
可靠性	幻觉率、拒答是否恰当
推理性能	首 Token 延迟、Tokens/s
运行成本	显存使用、吞吐量、单位请求成本

16.2 实际评估应该怎么做？

以一个客服微调项目为例：

先建立一批独立测试问题，例如 500 道，覆盖：

- 常规产品咨询
- 售后处理
- 多轮上下文
- 模糊问题
- 缺少知识时的回答
- 需要拒绝或升级人工处理的情况
- 固定格式输出

然后让基础模型和微调模型在相同输入、相同解码配置下回答，并进行自动测试与人工评测。

例如：

指标	基础模型	微调模型	结论
格式通过率	72%	96%	提升
任务回答正确率	78%	86%	提升
不支持的事实陈述率	9%	7%	改善
平均回答时延	1.2s	1.3s	略有增加

表格中的数字是教学假设，不是真实实验结果。

模型是否值得上线，应由整体业务目标决定，而不是单一 Loss 指标。

十七、第十三阶段：模型部署与上线

训练结束并不意味着流程结束。

一个典型的大模型应用部署流程是：

> 用户请求 / 客户端 → API Gateway / 业务后端 → 推理服务：vLLM / SGLang 等（加载模型、Tokenizer、可选 LoRA、处理批量请求）→ 模型生成结果 → 输出校验 / 安全控制 / 结果返回

例如使用 vLLM，支持 LoRA 的模型可以加载基础模型并启用适配器。

代码块

```
vllm serve Qwen/Qwen3-0.6B-Base \
  --enable-lora \
```

```
    --lora-modules tutor=./outputs/qwen3_lora
```

实际部署需要确保模型架构与 LoRA 适配器均受当前 vLLM 版本支持。

来源：vLLM

上线后还需要监控请求失败率、延迟、吞吐量、输出质量、安全事件以及数据和任务分布的变化。

如果线上效果退化，应先定位问题来自检索、提示词、模型、输入数据还是基础设施，而不是直接重新训练。

十八、训练中最常见的 10 类问题

问题	常见原因	排查方向
GPU OOM	序列太长、Batch 太大	减小 Micro Batch、降低长度、开启梯度检查点
Loss 不下降	学习率不合适、标签错误、数据质量差	检查数据、梯度和学习率
Loss 出现 NaN	数值不稳定、梯度过大	检查精度、裁剪梯度、降低学习率
训练 Loss 降而验证变差	过拟合、分布偏移	减少训练轮数、改善数据
模型重复用户问题	标签掩码或模板不正确	检查 Chat Template 与 Loss Mask
模型不停止输出	EOS/EOT 标记不匹配	核对 Tokenizer 与停止条件
模型回答乱码	Tokenizer 不匹配、训练失败	检查词表、权重与编码配置
微调后通用能力下降	灾难性遗忘、数据过窄	保留通用评估集，调整训练数据
模型只会背答案	数据太少、重复太多	提高样本多样性和泛化测试覆盖
多 GPU 效率低	通信、数据加载或负载失衡	分析 GPU 利用率、吞吐量和网络通信

对于刚入门的人，最值得养成的习惯是：

每次正式训练前，先尝试让模型在一个很小的数据集上成功过拟合。

这个测试可以帮助你发现训练代码是否真正具备更新参数、计算正确 Loss、保存加载权重的能力。

小数据过拟合通过后，再扩大数据集、验证泛化能力。

十九、建议的完整学习路线

如果想从入门提升到能独立实施大模型训练项目，推荐按下面的顺序学习。

大模型训练工程师学习清单 · 已完成 0 / 8 个阶段 · 0%

- [ ] **第 1 阶段：PyTorch 基础** — 理解 Tensor、自动求导、反向传播、优化器，并手写一个训练循环。
- [ ] **第 2 阶段：Transformer 原理** — 理解 Embedding、Attention、MLP、Mask、Decoder-only。
- [ ] **第 3 阶段：从零小模型预训练** — 训练 Tokenizer，随机初始化 GPT，完成训练和保存。
- [ ] **第 4 阶段：监督微调** — 掌握 Chat Template、SFT 数据、Loss Mask 和验证。
- [ ] **第 5 阶段：LoRA / QLoRA** — 在一张 GPU 上完成已有语言模型的参数高效微调。
- [ ] **第 6 阶段：分布式训练** — 学习 DDP、FSDP、ZeRO、混合精度和断点恢复。
- [ ] **第 7 阶段：偏好优化** — 使用偏好数据研究 DPO，再进一步探索强化学习。
- [ ] **第 8 阶段：模型评估和服务化** — 建立测试集、部署推理服务、监控实际效果。

二十、最后把整个训练流程串起来

可以把完整训练工程理解为下面这条主线：

顺序	阶段	主要产出
1	明确目标和资源预算	模型训练方案
2	收集、清洗、去重和划分数据	高质量训练数据
3	训练或选择 Tokenizer	词表和文本编码工具
4	选择 Transformer 架构	模型结构配置
5	初始化参数	初始模型
6	预训练	Base Model
7	领域继续预训练（可选）	领域适应模型
8	SFT / LoRA / QLoRA	任务或指令模型
9	DPO / 强化学习（可选）	偏好优化后的模型
10	离线评估及安全测试	评估报告
11	推理部署和接口集成	模型服务
12	线上监控、数据回流和迭代	持续改进的模型系统

需要记住的五个核心结论

1. 预训练与微调在底层训练机制上有很大共性：都涉及数据、前向传播、损失计算、梯度更新，只是初始权重、数据形式和优化目标不同。
2. 数据质量非常关键：高质量、有代表性且合规的数据，比盲目增加低质量数据更重要。
3. LoRA 是非常适合工程实践的入门方式：无需从零训练大模型，就能学习和验证完整的微调流程。
4. 模型 Loss 下降不等于业务效果提升：一定要建立独立评估体系。
5. 完整的大模型工程包含训练、对齐、评估、部署和监控，不只是调用一次 trainer.train()。